

ECOLE NORMALE SUPÉRIEURE DE L'ENSEIGNEMENT
TECHNIQUE DE MOHAMMEDIA
UNIVERSITÉ HASSAN II DE CASABLANCA



المدرسة العليا للأساتذة التعليم التقني
المحمدية
جامعة الحسن الثاني بالدار البيضاء

**Département Mathématiques
& Informatique
1ère MSDIA**

**Algorithmique
&
Structures de données
&
Programmation
en Python**

Objectifs

☞ **Comprendre les bases algorithmiques et Apprendre à utiliser les techniques de résolutions de problèmes informatiques :**

- **Analyse descendante/ascendante (abstractions successive),**
- **Paradigmes de résolution de problèmes:**
Brute force, decrease/transform/divide and conquer, Greedy, Dynamic programming, backtracking, Graph algorithms, randomized algorithms, ...
- **Formulation récursive/itérative,**
- **Structure de contrôle, Procédure/Fonction/Module,**
- **Structures de données : linked lists, stacks, queues, sets, hash tables, trees, heaps, priority queues, and graphs,**

Objectifs

- ➡ **Savoir appliquer les techniques d'analyse d'algorithmes pour évaluer leurs efficacités :**
 - **Analyse asymptotique,**
 - **Analyses du Cas pire, du cas meilleur, du cas moyen**
- ➡ **Apprendre à coder des algorithmes en Python**
- ➡ **Apprendre à élaborer des applications programmées en Python**

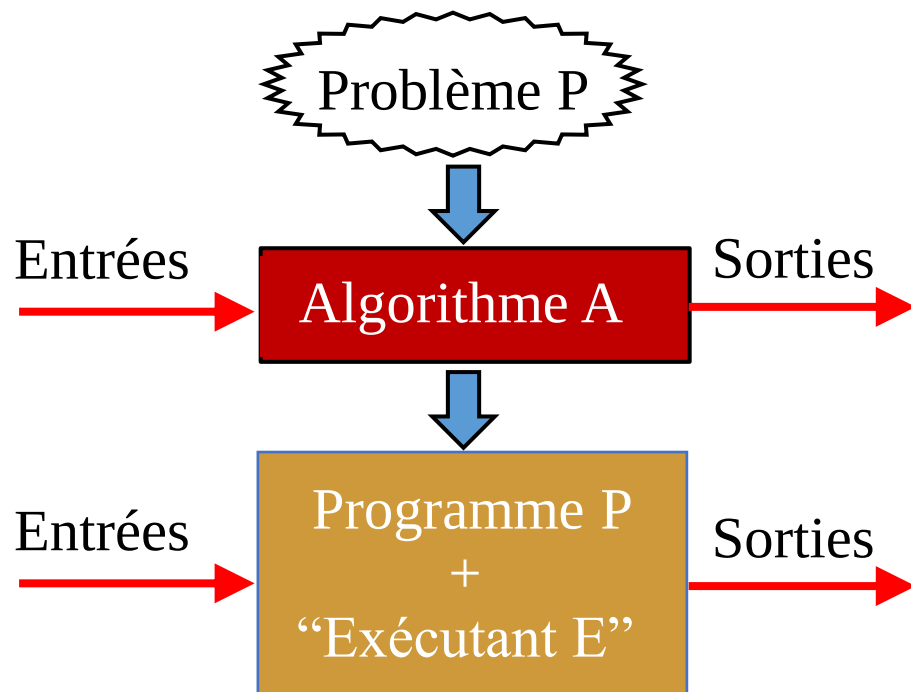
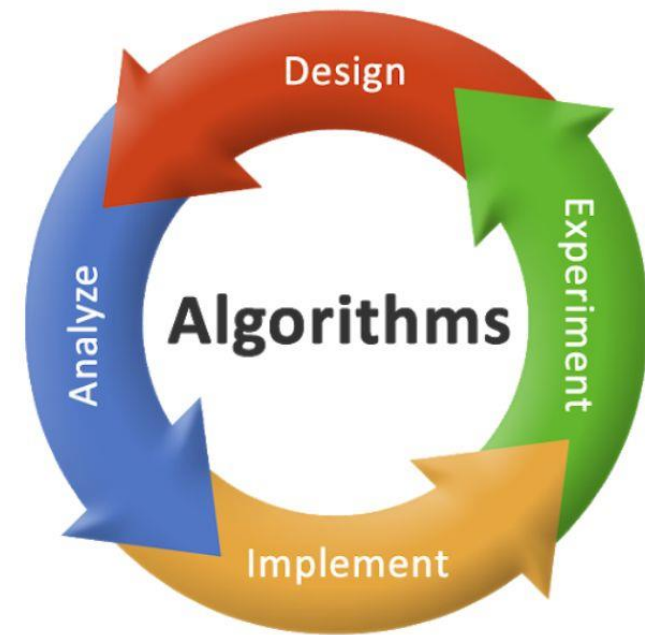
Notions abordées

- ❑ **Notions de Algorithme / Algorithmique**
 - **Validité d'un algorithme**
 - Preuve de sa correction (postconditions, préconditions, invariants)
 - **Analyse d'un algorithme**
 - Complexité d'un algorithme (coûts en temps et en espace)
 - **Structures de données**
- ❑ **Notion de Programme/Programmation**
- ❑ **Conception d'un algorithme**
- ❑ **Exemples d'algorithmes**
- ❑ **Structure d'un algorithme**

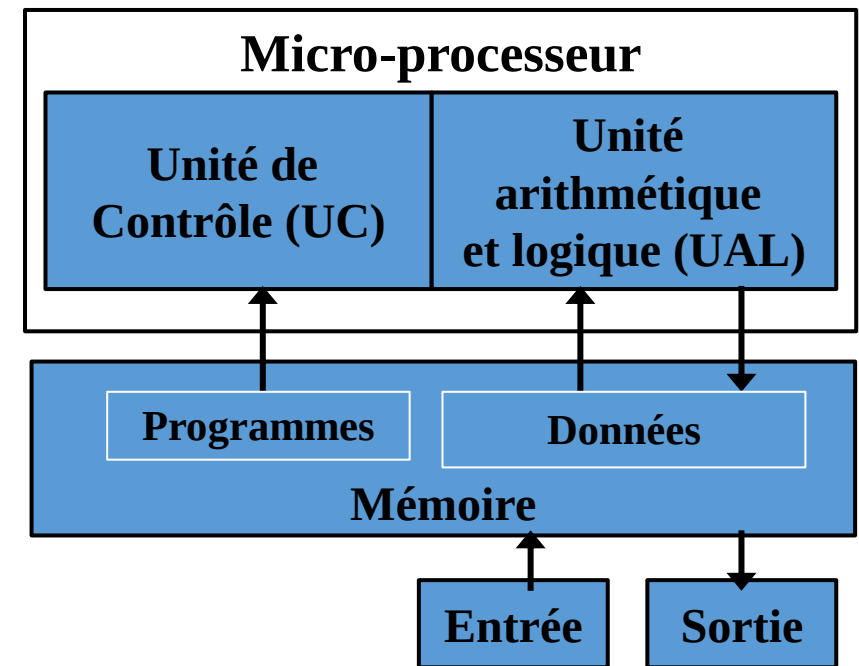
1. Introduction

Principales Phases du génie logiciel :

- La conception,
- L'analyse (justesse, performance, ...),
- Implémentation,
- Tests.



Exécutant (Von Neumann)



2. Notion d'algorithme/d'algorithmique

Algorithme : Un algorithme est une **suite finie et ordonnée d'instructions ou d'étapes** visant à résoudre un **problème spécifique**.

Chaque étape doit être **exprimée sans ambiguïté** dans un **langage formel** et doit être **réalisable par un exécutant** permettant une **exécution automatique**.

Il s'agit d'une **méthode abstraite** qui **prend des données en entrée**, **effectue un traitement spécifique**, et **produit un résultat en sortie**.

Un **algorithme séquentiel** effectue des **opérations une à une**, en **séquence**,

Un **algorithme parallèle** ou **distribué** peut effectuer **plusieurs opérations simultanément**.

Algorithmique : Discipline qui s'intéresse à l'**art de construire des algorithmes** ainsi qu'à caractériser leur **validité**, leur **robustesse**, leur **réutilisabilité**, leur **complexité** ou leur **efficacité**.

Caractéristiques d'un algorithme

Un algorithme doit avoir les caractéristiques suivantes ::

- **Clair et sans ambiguïté** : Chacune de ses étapes doit être claire et ne doit avoir qu'à un seul sens.
- **Entrées bien définies (zéro ou plusieurs entrées)**
- **Sorties (résultats) bien définies** : les sorties produites (au moins une) doivent être également être bien définie.
- **Faisable** : l'algorithme doit être simple, générique et pratique, de manière à pouvoir être exécuté avec les ressources disponibles.
- **Indépendant du langage** : l'algorithme conçu doit être indépendant du langage.

- **Précision** : toutes les instructions d'un algorithme doivent être sans ambiguïté, précises et faciles à interpréter.
- **déterministe** => donnant le même résultat pour le même cas d'entrée.
- **Finitude** : un algorithme doit se terminer après un nombre fini d'étapes dans tous les cas de test. Les **boucles infinies** ou les **fonctions récursives sans conditions de base** ne possèdent pas de finitude.
- **Efficacité** : doit coûter un minimum de ressources (en temps et en espace)

Questions de base liées aux algorithmes

- **Comment concevoir des algorithmes?**
- **Comment exprimer des algorithmes?**
- **Prouver l'exactitude?**
- **Analyse d'efficacité (ou de complexité)?**
 - **Analyse théorique**
 - **Analyse empirique**
- **Etudier l'Optimalité?**

3. Notion de Programme/Programmation

Programme :

Une ne **séquence d'instructions** destinées à être **exécutées par un ordinateur**.

Code source :

Est un programme exprimé dans un **langage de programmation** compréhensible par un être humain.

Un algorithme peut être implémenté dans différents langages de programmation : C, C++, Fortran, Java, Python, ...

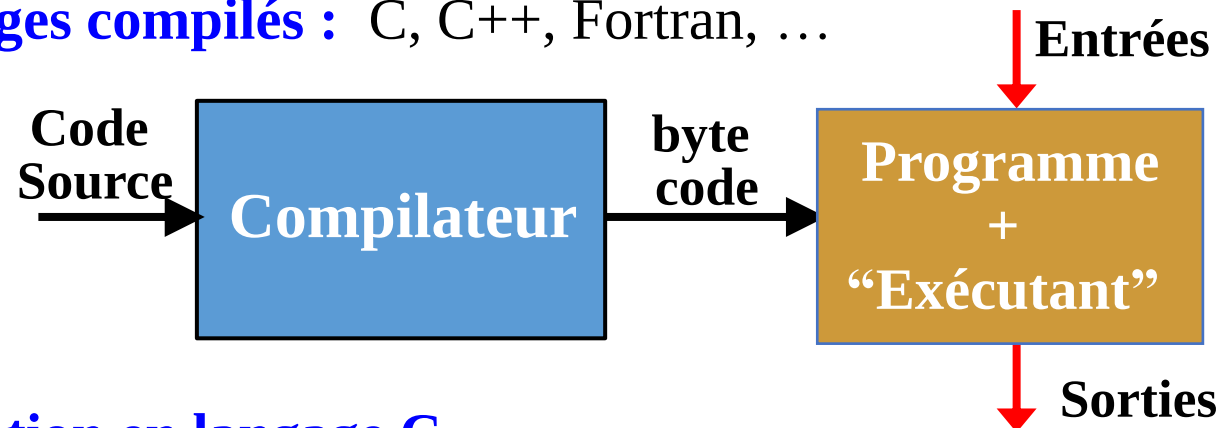
Code binaire :

Programme exprimé **en langage machine**, pouvant être directement exécuté par l'ordinateur.

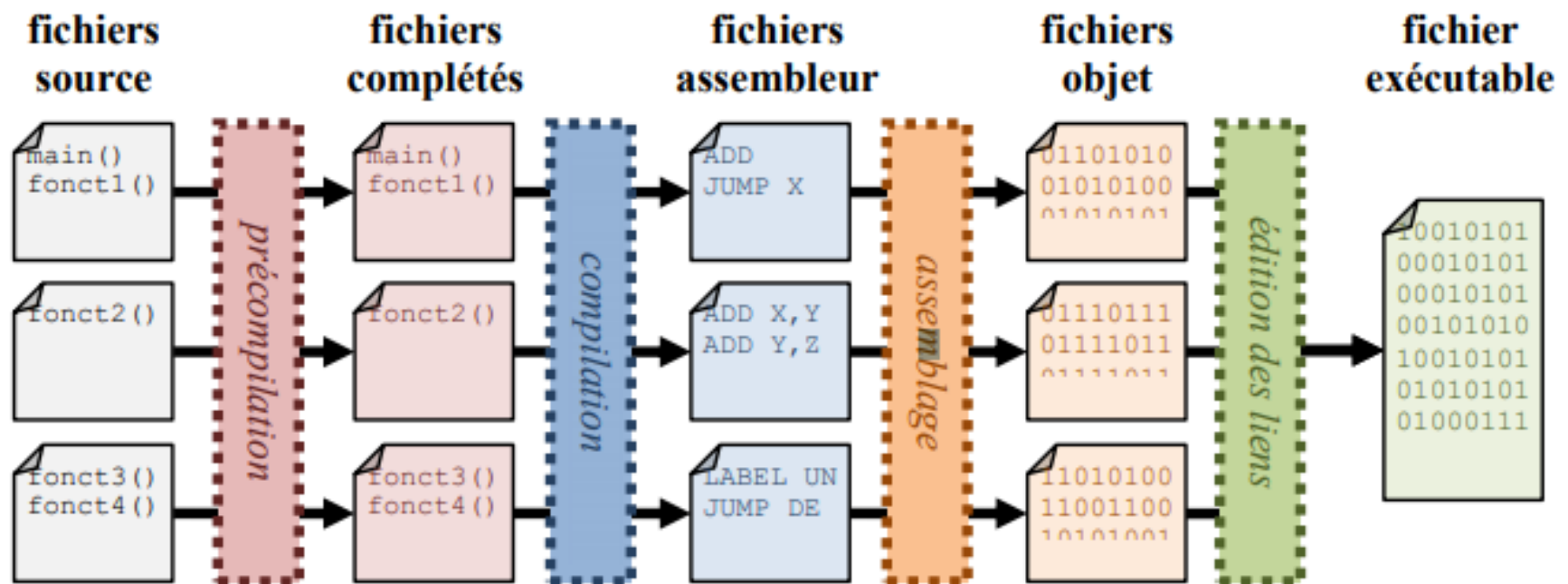
Compilateur

Programme qui permet de **transformer** un **code source** en **code binaire**, de manière à obtenir un **fichier exécutable** à partir d'un **fichier source**

Exemple de langages compilés : C, C++, Fortran, ...

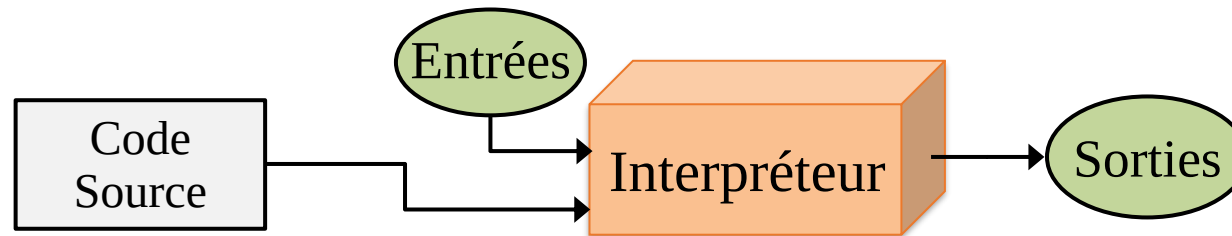


Etapes de compilation en langage C

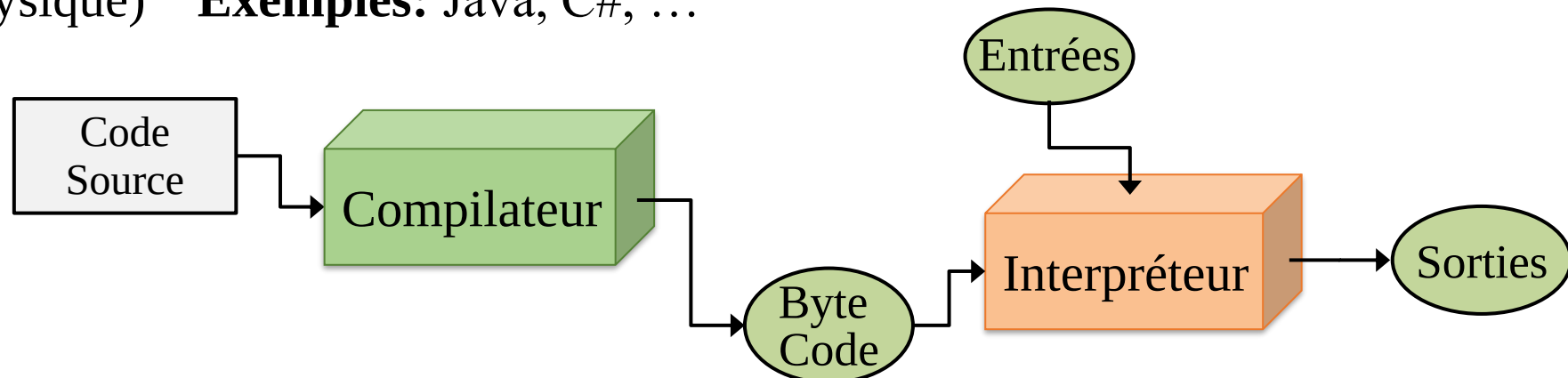


Interpréteur : permet de traduire et d'exécuter, instruction par instruction, les instructions du programme source, autrement les instructions, l'une après l'autre, sont lues, analysées et préparées pour l'exécution par le processeur

Exemples: Lisp, Prolog, Perl, Python, ...



Semi-compileur : Basé sur un mécanisme qui combine les deux techniques (compilation pour une machine virtuelle et interprétation pour la machine physique) **Exemples:** Java, C#, ...



4. Types de problèmes

- **Tris d'éléments d'une liste**
- **Recherche d'un élément dans une liste**
- **Calcul sur des chaînes (caractères, nombres, bits, ...)**
- **Problèmes de graphes**
- **Problèmes combinatoires**
- **Problèmes géométriques**
- **Problèmes numériques, ...**

4. Techniques de conception de solutions

- **Force brute :**

Principe : Essayer toutes les combinaisons possibles pour résoudre le problème, sans aucune optimisation pour des problèmes de petite taille.

Exemple : Recherche d'un motif dans une chaîne.

- **Divide| Decrease| Transform and Conquer :**

Principe : Diviser un problème en sous-problèmes plus petits, les résoudre de manière indépendante, puis combiner les solutions pour résoudre le problème global.

Exemples : tri rapide ou le tri fusion recherche binaire.

- **Greedy (glouton)**

Principe : Faire des choix locaux optimaux à chaque étape dans l'espoir de trouver une solution globale optimale. Il ne regarde pas au-delà de l'étape courante.

Exemples : Algorithme de Kruskal ou Prim pour les arbres couvrants minimums, problème du sac à dos.

■ **Dynamic Programming**

Principe : Résoudre des sous-problèmes qui se chevauchent en les mémorisant pour éviter de les recalculer plusieurs fois. Elle décompose les problèmes en sous-problèmes optimaux et les résout de manière itérative ou récursive.

Exemples : Problème du sac à dos dans sa version entière, algorithme de Floyd-Warshall pour les plus courts chemins entre tous les nœuds.

■ **Backtracking (Retour sur trace)**

Principe : Explorer toutes les solutions possibles à un problème en construisant une solution partielle et en revenant en arrière (backtrack) lorsque cette solution ne mène pas à une solution correcte.

Exemples : Résolution de puzzles comme le Sudoku, problème des N Reines.

■ **Branch and Bound :**

Principe : Extension du backtracking avec l'ajout de bornes pour éliminer rapidement certaines solutions possibles qui ne peuvent pas mener à une solution optimale.

Exemples : Résolution du problème du voyageur de commerce, problème du sac à dos.

■ Randomized Algorithms (Algorithmes randomisés)

Principe : Utiliser des décisions aléatoires dans certaines étapes de l'algorithme pour obtenir une solution avec une forte probabilité de succès, souvent plus rapide que les méthodes déterministes.

Exemples : Algorithme de Monte Carlo, algorithme de QuickSort randomisé.

■ Greedy with Local Search (Recherche locale)

Principe : Partir d'une solution initiale et la modifier légèrement de manière itérative pour améliorer progressivement la solution jusqu'à ce qu'on ne puisse plus trouver de meilleure solution locale.

Exemples : Problèmes d'optimisation dans les systèmes complexes, tels que le problème de la satisfaction des contraintes (CSP).

■ Graph Algorithms (Algorithmes sur les graphes)

Principe : Les algorithmes sur les graphes sont utilisés pour résoudre des problèmes où les relations entre objets peuvent être représentées sous forme de graphes (sommets reliés par des arêtes).

Exemples : Algorithme de Dijkstra pour les plus courts chemins, parcours en profondeur (DFS) et en largeur (BFS).

5. Démarche méthodologique

5.1. Besoin de méthode

Pour concevoir une application on doit recourir à une **méthode** pour bien aborder la complexité de la **structure de données** et des **traitements à effectuer** ...

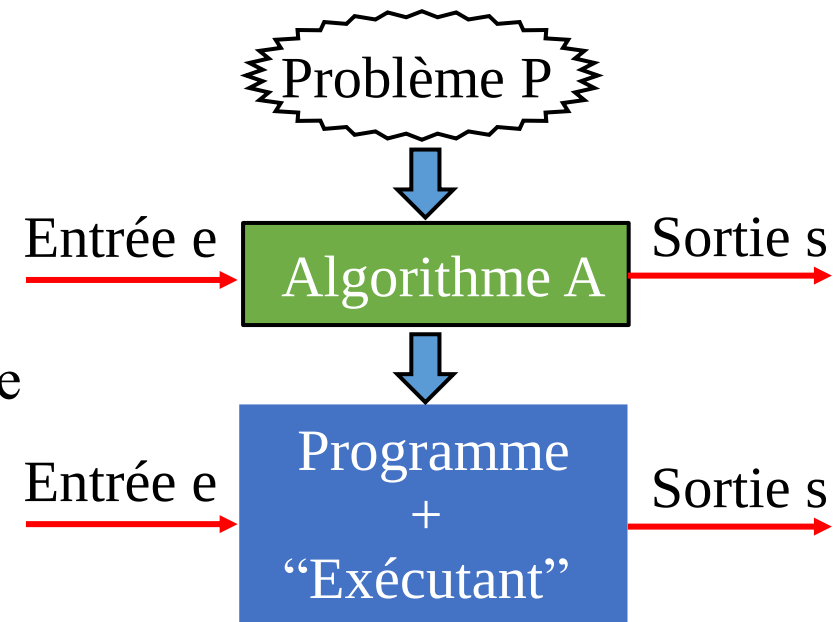
5.2. Composantes d'une méthode de développement

Une méthode de développement est composée des éléments suivants :

☞ **Stratégie de résolution :**

- Des **principes** : **abstraction** par niveaux, **diviser/réduire/transformer** pour **régner**
- **Approches de résolution itérative (incrémentale)**
- **Approche de résolution récursive**
- Une **démarche** : Étapes de mise en œuvre

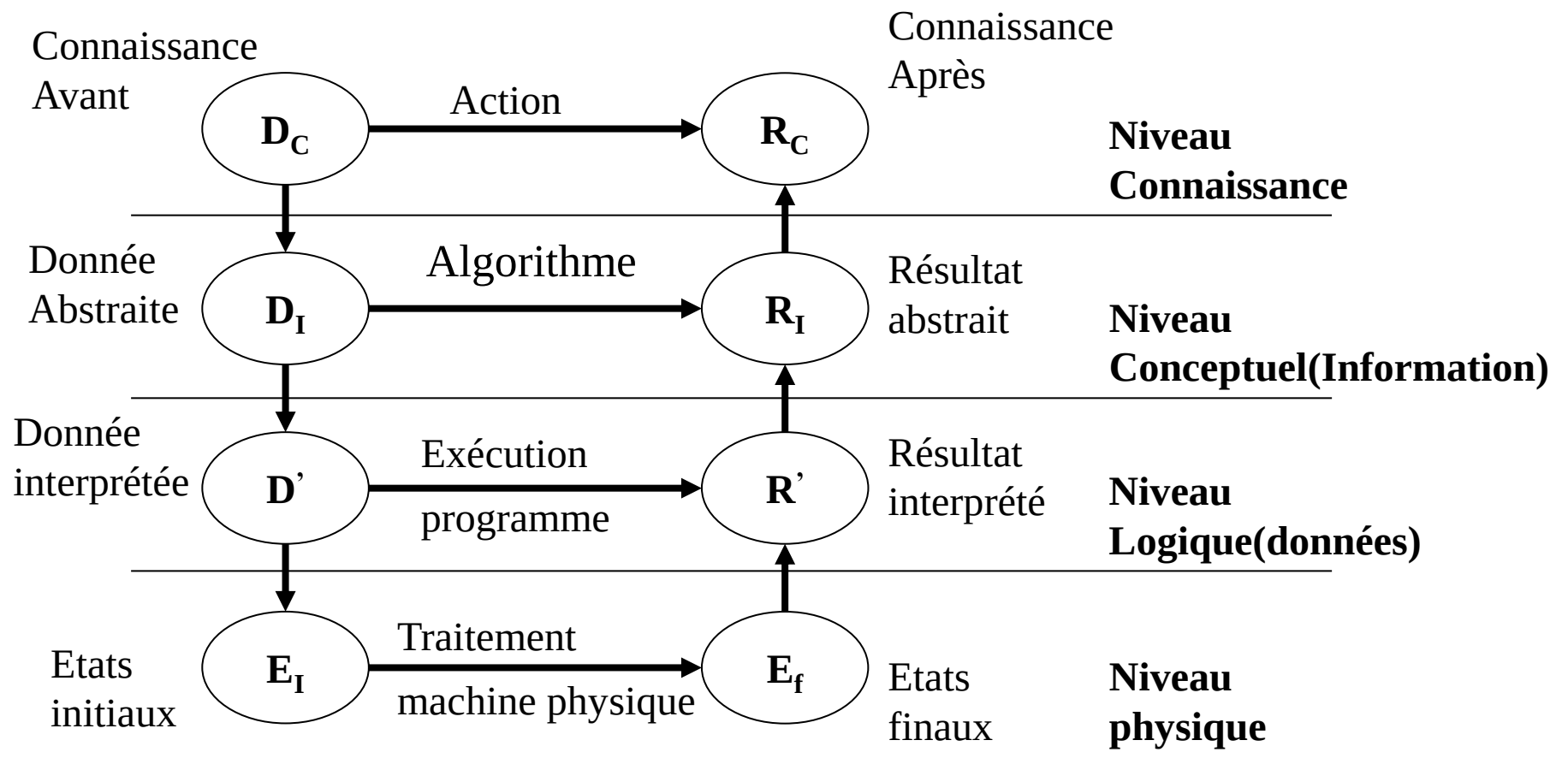
☞ Des **outils** : **Langages , modèles, ...**



a. abstraction

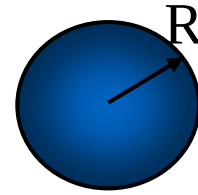
Le développement d'une application consiste à représenter les éléments de connaissances à des **niveaux d'abstraction différents** pour descendre du niveau **humain** au niveau de la **machine**

On distingue les quatre niveaux d'abstraction suivants :



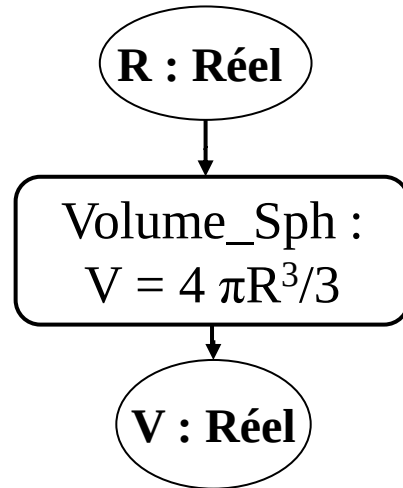
Abstraction - Exemple

Calcul du volume d'une sphère



$$V = 4 \pi R^3 / 3$$

**Niveau
Connaissance**



Algorithme Vol_Sph

Const Reel Pi = 3.14

Reel R, V

Lire(R)

$V \leftarrow 4 * \text{Pi} * R * R * R / 3$

Ecrire("Volume = ",V)

Fin

**Niveau
information**

/ code C*/*

```
#include<stdio.h>
```

```
const float Pi = 3.14 ;
```

```
float R, V;
```

```
int main(){
```

```
    scanf("%f", &R );
```

```
    V = 4 * Pi * x*x*x /3 ;
```

```
    printf(" volume : %.3f\n",V)
```

```
    return 0;
```

```
}
```

code python

```
import math
```

```
R=float(input())
```

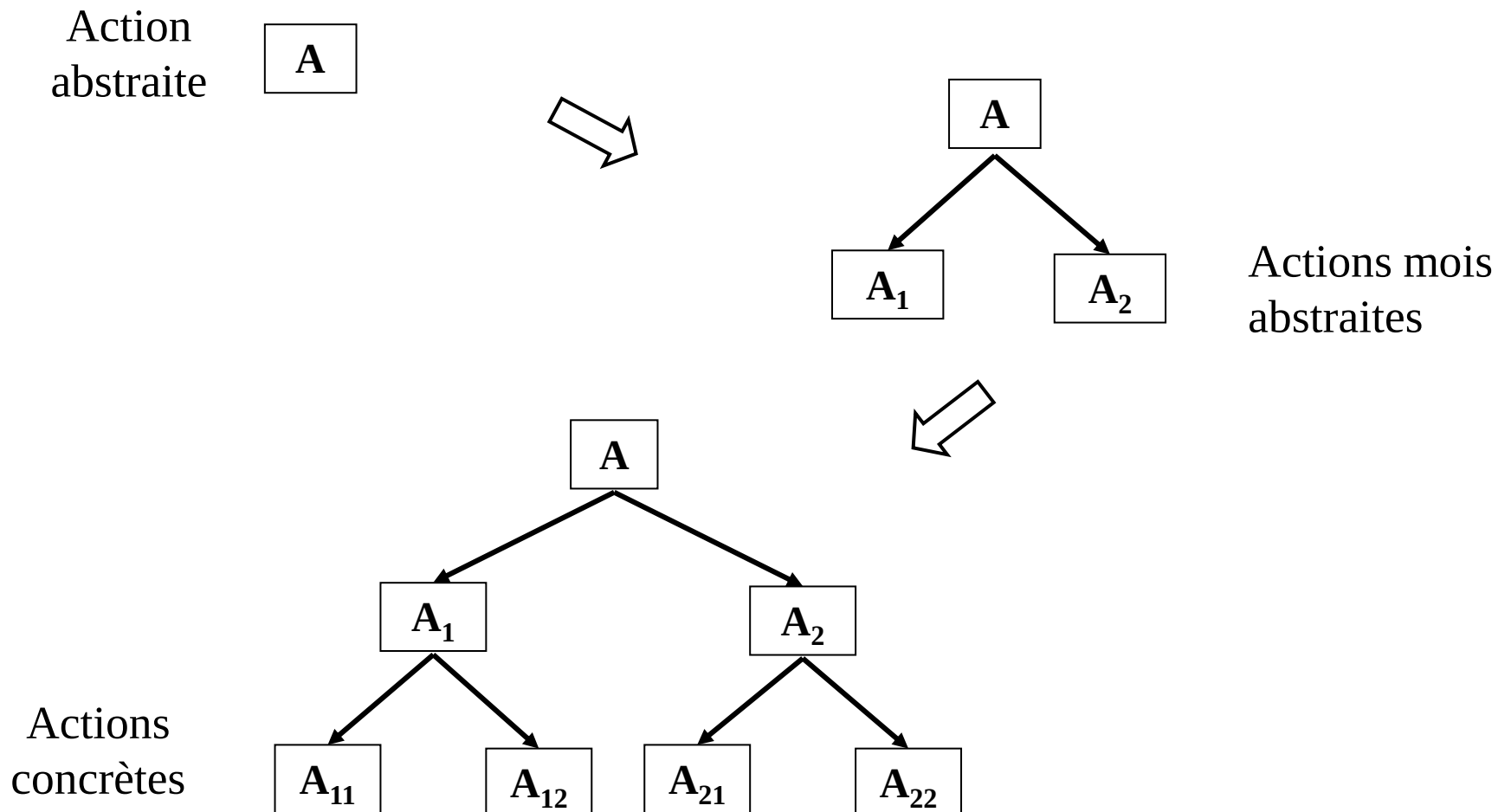
```
V = 4 * math.pi * R**3 /3
```

```
print(" volume :",V)
```

**Niveau
Logique**

b. Diviser pour régner : Analyse descendante

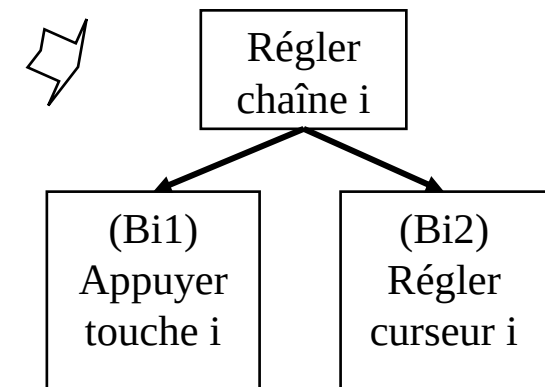
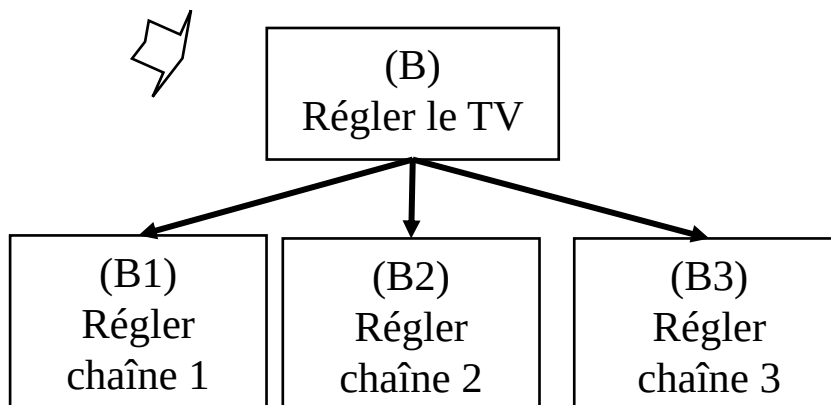
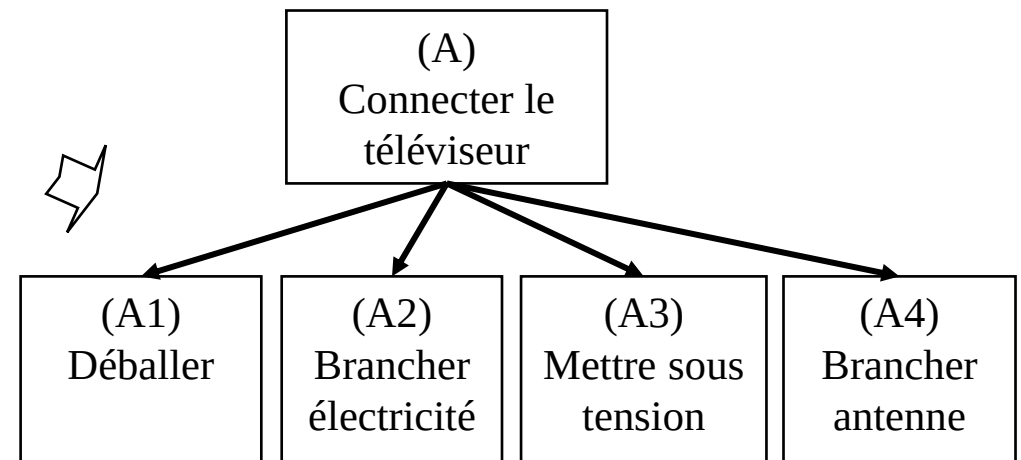
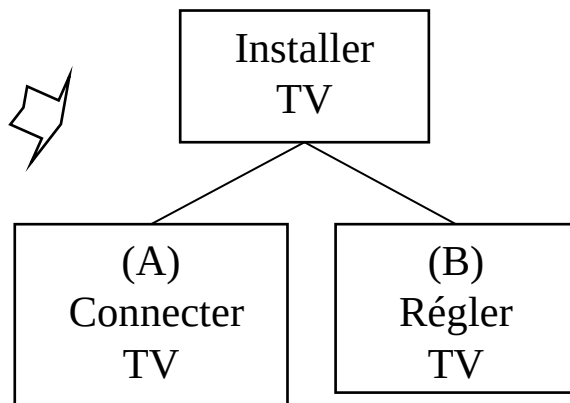
Diviser pour régner consiste à **décomposer** le problème complexe à résoudre en plusieurs **sous problèmes** moins complexes. A refaire cette décomposition sur les sous problèmes jusqu'à obtenir des sous problèmes faciles à résoudre.



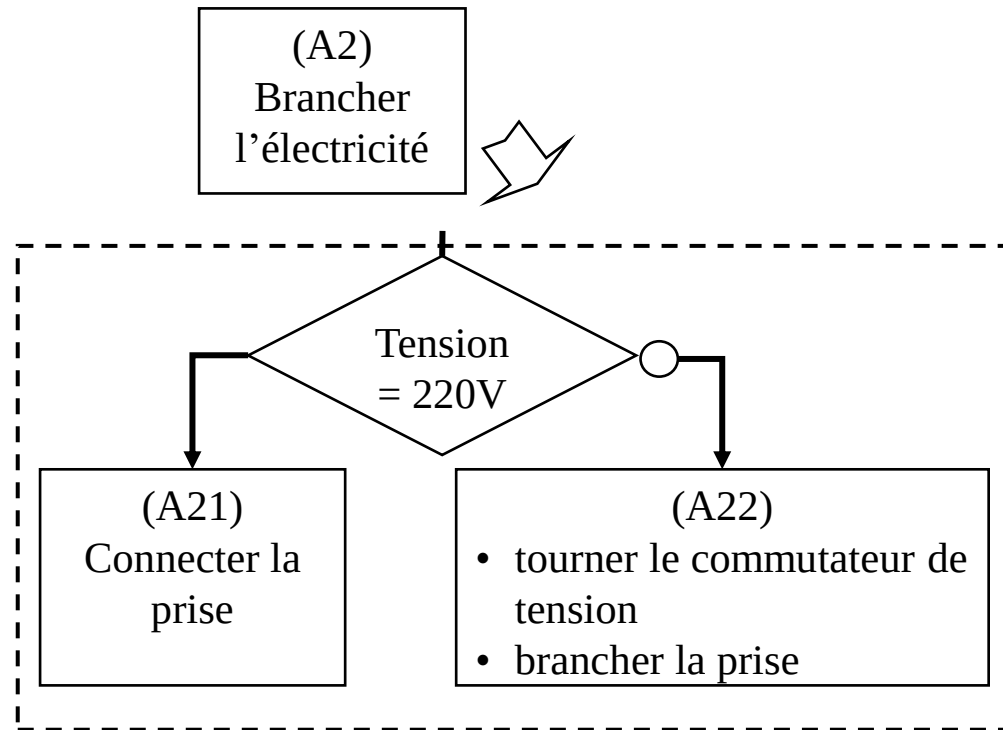
Exemple1 : Installation d'un téléviseur

Installer
TV

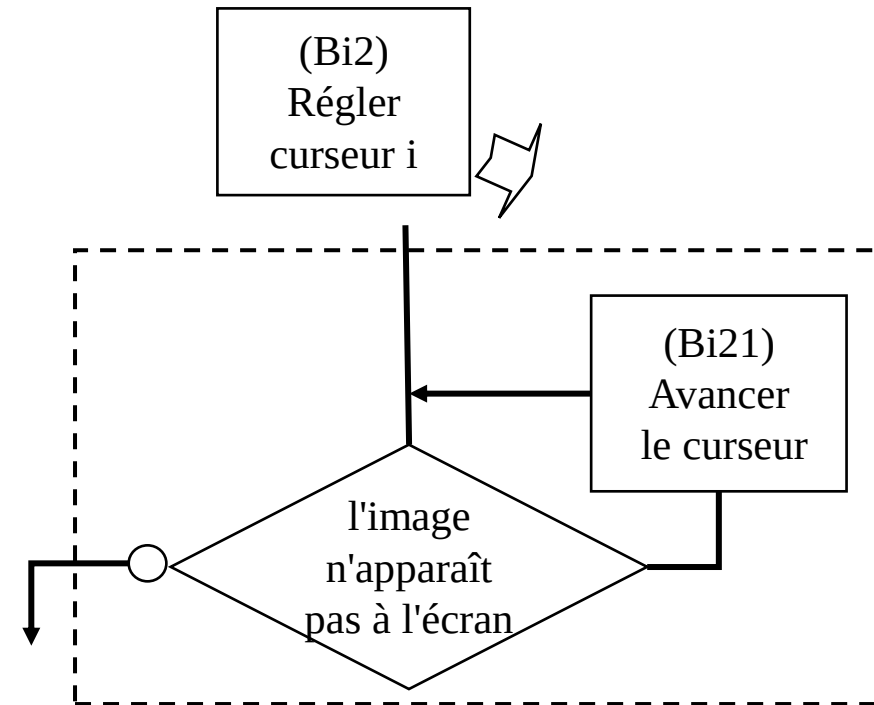
Enoncés séquentiels



Enoncé Conditionnel



Enoncé répétitif



Exemple 2 :

Vérifier si un nombre entier naturel x est un nombre **d'Armstrong**.

Soit x un nombre entier qui s'écrit dans la base 10 sous la forme $x = d_{n-1} \dots d_1 d_0$

$d_i \in \{0, 1, \dots, 9\}$ étant le digit numéro i , n étant la longueur du mot $\text{str}(x)$ ($n = |\text{str}(x)|$).

x est un nombre d'Armstrong ssi :

$$x = \sum_{i=0}^{n-1} d_i^n$$

Chap. I : Introduction à l'algorithmique

Les entrées :

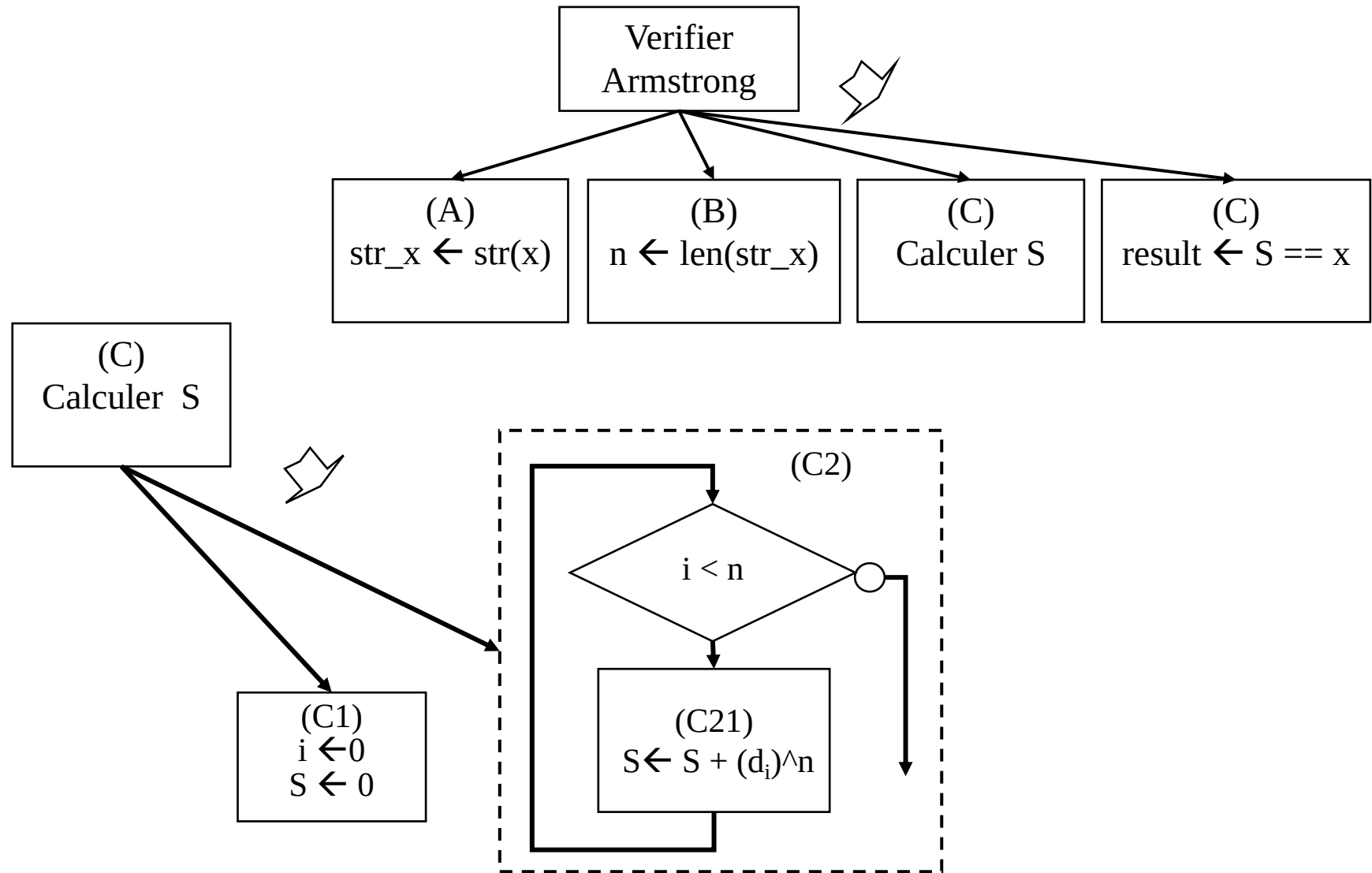
- x : un nombre d'Armstrong

Les sorties :

- $result$: Booléen;

Données intermédiaires :

- str_x : conversion de x en chaîne
- n : longueur de str_x
- d_i : digit n° i du nombre x
- S : Somme des d_i^n , $i=0..n-1$



Conclusion

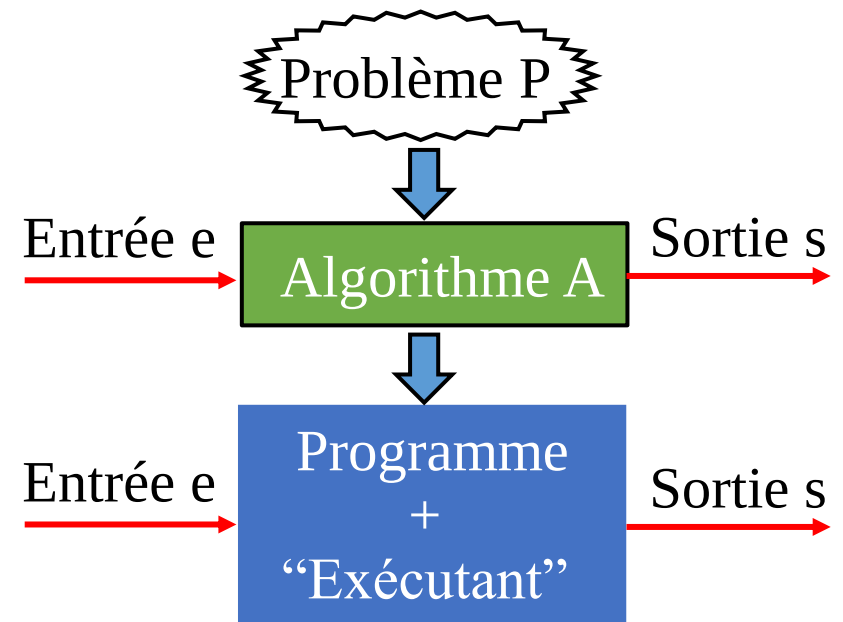
La solution à un problème bien défini peut être formulée comme une suite des trois énoncés suivants :

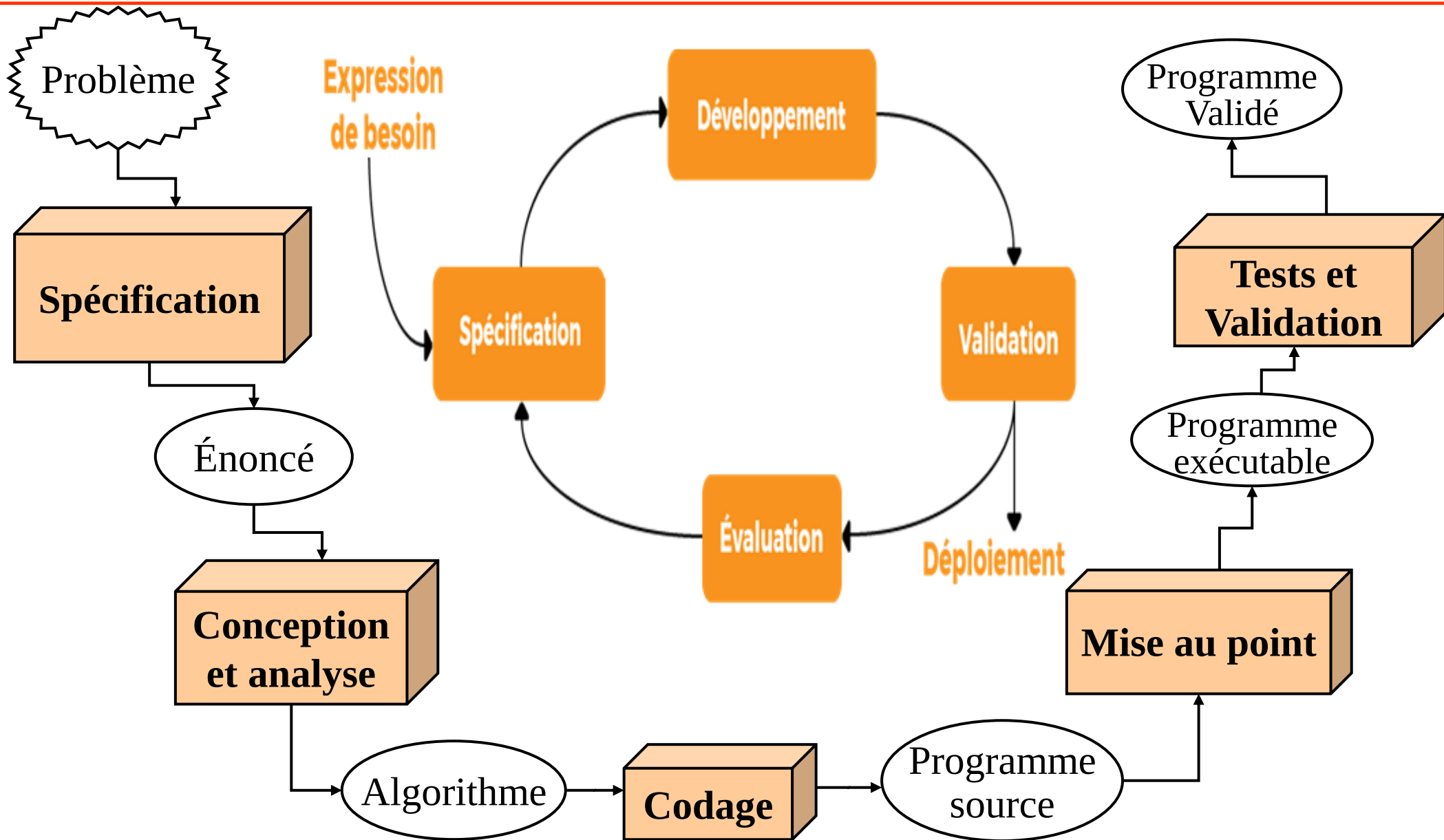
- **Séquentiel** : suite d'étapes
- **Conditionnel** : choix entre deux étapes suivant une condition
- **Répétitif** (itératif) : répétition d'une étape

c. Démarche

la résolution d'un problème de calcul passe par différentes étapes :

1. Comprendre et définir le problème
2. Identifier et caractériser les entrées et sorties e et s
3. Construire un modèle global du problème
4. Définir la stratégie de résolution
5. Concevoir et formuler le schéma de résolution
6. Vérifier l'exactitude de l'algorithme
7. Analyser l'algorithme
8. Implémenter l'algorithme
9. Tester le programme
10. Faire la documentation



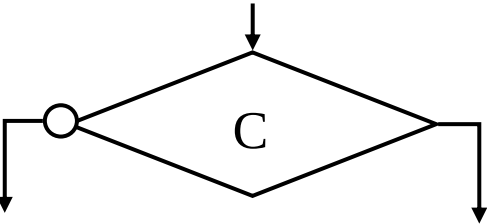
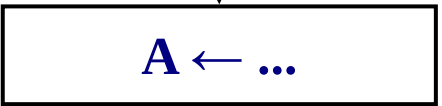
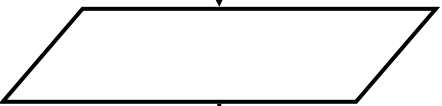
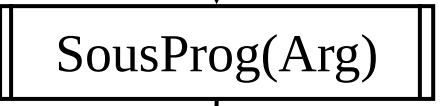
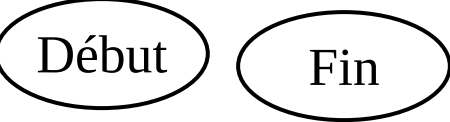


d. Outils

Langages : Outil textuel pour écrire des algorithmes (pseudo code = français)

Organigramme

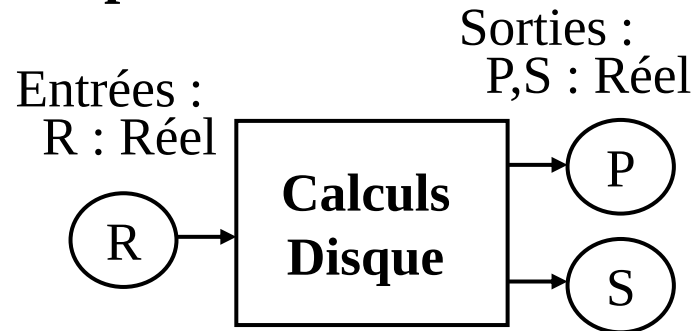
Représentation schématique utilisant des **symboles graphiques** pour décrire les étapes de résolution d'un problème :

	Marque le passage d'une action à une autre
	Rupture de séquence marquant un choix conditionnel d'exécuter une action suivant la valeur de la condition C
	Action de calcul pour modifier la valeur d'une variable
	Action d'entrée(lecture d'une donnée)/Sortie (affichage d'un résultat)
	Action d'appel à un sous programme
	Marque le début et la fin de traitement

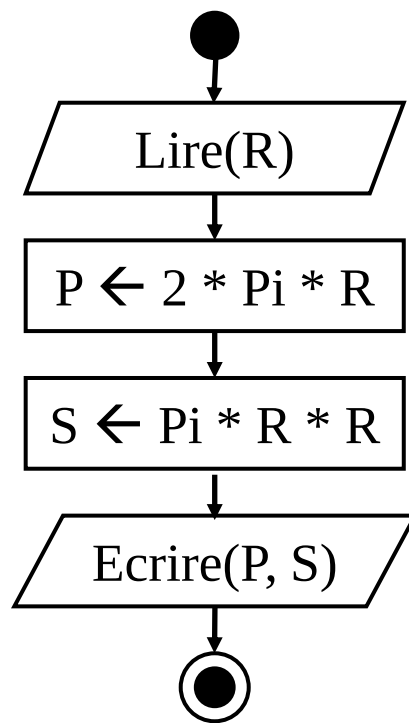
6. Exemples d'algorithmes

Algorithme1 : Calcul du périmètre P et de la surface S d'un disque de rayon R .

Spécification



Organigramme



Algorithme Disque

/ déclarations */*

Const Réel $\text{Pi} = 3.14$

Réel R, P, S

/ Saisie */*

Lire(R)

/ Calculs */*

$P \leftarrow 2 * \text{Pi} * R$

$S \leftarrow \text{Pi} * R * R$

/ Affichage */*

Ecrire("Périmètre = ", P)

Ecrire("Surface = ", S)

Fin

#Program Disque.py

from math import pi

Saisie

R=float(input())

/ Calculs */*

$P = 2 * \text{pi} * R$

$S = \text{pi} * R^{**}2$

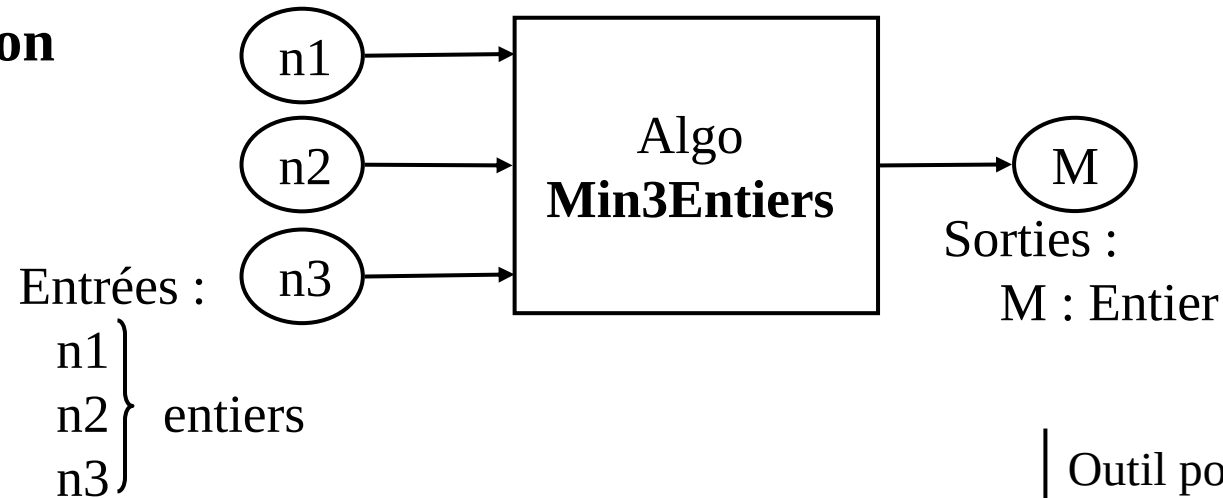
/ Affichage */*

print("Périmètre = ", P)

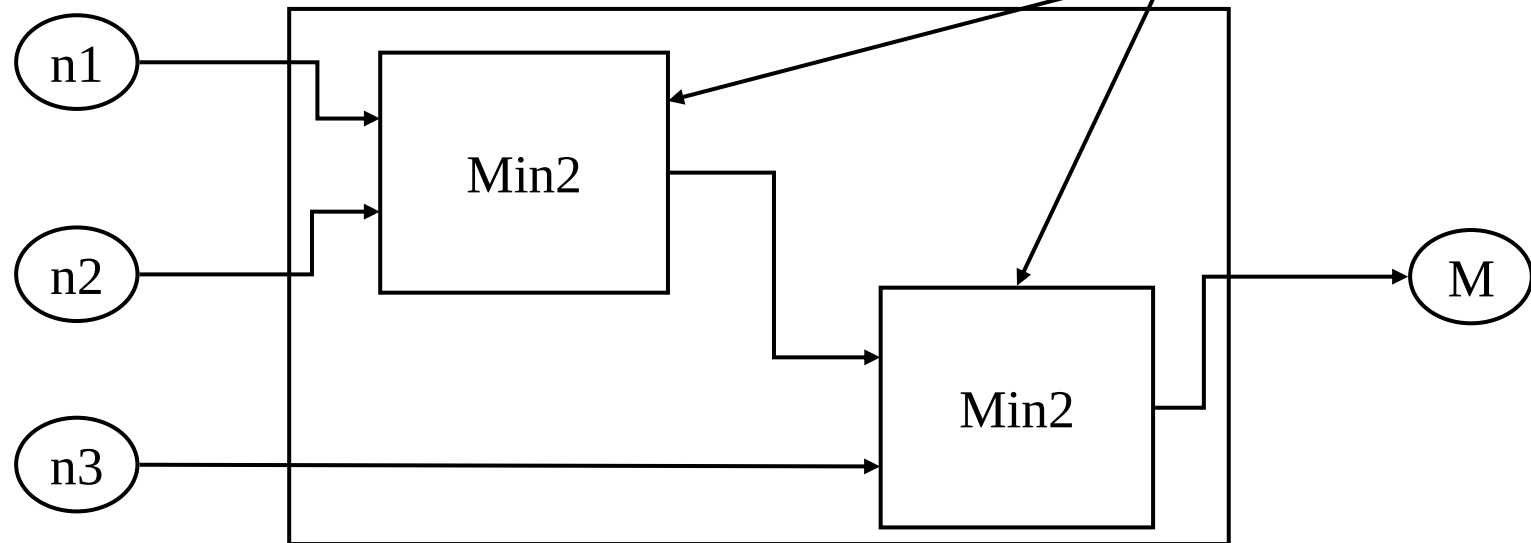
print("Surface = ", S)

Algorithme2 : recherche du plus petit parmi trois nombres entiers $n1$, $n2$ et $n3$.

Spécification

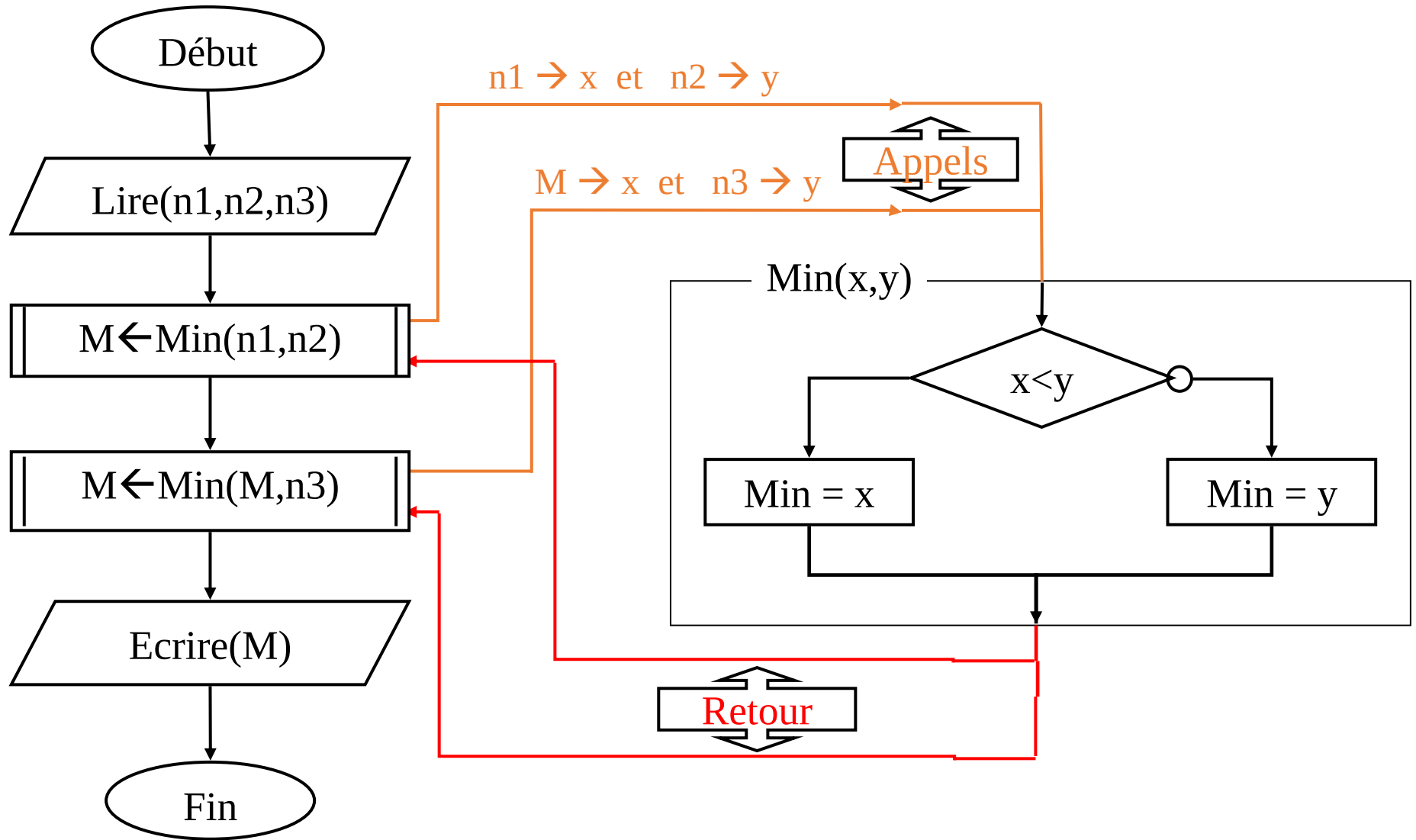


Traitements



Remarque : Le problème se trouve simplifié au calcul du minimum de 2 entiers.

Organigramme



Algorithme

```
/* Définition fonction Min */  
Entier Min(Entier x , Entier y)  
    Si (x<y)  
        Retourner x  
    Sinon  
        Retourner y  
FinSi  
Fin
```

Algorithme Min3

```
/*déclarations */  
    Entier n1 , n2 ,n3 , M  
/* Corps principal */  
    /*Saisie des valeurs de n1, n2 et n3 */  
    Ecrire("Introduire les nombres : ")  
    Lire(n1 , n2 , n3)  
    /* mettre dans M le Min de n2 et n3*/  
    M ← Min(n1,n2); /* Appel de Min */  
    /* mettre dans M le Min de M et n3*/  
    M ← Min(M,n3)  
    /* afficher M */  
    Ecrire("Le minimum = " , M)  
Fin
```

Programme Python

```
#Programme Min3.py  
  
#Saisie des valeurs de n1, n2 et n3  
n1 = int(input("Introduire n1 : "))  
n2 = int(input("Introduire n2 : "))  
n3 = int(input("Introduire n3 : "))  
  
#min est prédéfinie dans python  
M = min(n1,n2,n3)  
  
#afficher M  
print("Le minimum = " , M)
```


Chap. I : Introduction à l'algorithmique

7. Structure d'un algorithme / Programme Python

Un algorithme est structuré en trois parties :

- Un **en-tête** : Pour nommer l'algorithme
- Des **déclarations/Définitions** : Pour préciser les **données** et les **modules** (**fonctions** et **procédures**) utilisés par l'algorithme
- Un **corps principal** : contient les instructions à exécuter par l'algorithme

```
/* définition de fonctions/Procédures*/  
type nom_fonction(liste_paramètres)  
    ...  
Fin  
...  
Algorithme Nom_Algo  
    /* déclarations (types/variables/const/procédures/fonctions)*/  
    ...  
    /* instructions */  
    ...  
Fin
```

Chap. I : Introduction à l'algorithmique

Un programme Python est structuré en trois parties :

- Des **Importation des modules**
- Des **Définitions de fonctions**
- Un **corps principal** : contient les instructions à exécuter

```
# Importation des modules
import module_name

# Définition des fonctions
def nom_fonction(list_params):
    # Instructions
    ...
    pass

...

# Corps principal du programme
if __name__ == "__main__":
    # Instructions, Appel des fonctions et logique du programme
    ...
    pass
```